

Experiment Title 7: Java Collection Framework

Objective: The objective of this experiment is to understand the concept of the Java Collection Framework, learn how to store, retrieve, and manipulate data using collection classes, demonstrate the usage of commonly used interfaces and classes such as List, Set, Map, ArrayList, HashSet, and HashMap, and compare the advantages of using collections over arrays.

The Java Collection Framework (JCF) is a set of classes and interfaces in Java that provides ready-made data structures (like lists, sets, and maps) to store, manage, and manipulate groups of objects efficiently.

Feature	List	Set	Map
Definition	Ordered collection of elements	Unordered collection of unique elements	Collection of key-value pairs
Duplicates	Allowed	Not allowed	Keys – not allowed, Values – allowed
Order	Maintains insertion order	Does not guarantee order (HashSet) Some maintain order (TreeSet, LinkedHashSet)	Keys are unique; order depends on implementation (HashMap, TreeMap, LinkedHashMap)
Access by Index	Yes (e.g., list.get(0))	No	Access by key (e.g., map.get(key))
Examples	ArrayList, LinkedList	HashSet, TreeSet	HashMap, TreeMap

Key Points

- Provides dynamic data structures (unlike arrays, which are fixed size).
- Main interfaces: Collection, List, Set, Queue, Map.
- Common classes: ArrayList, LinkedList, HashSet, TreeSet, HashMap.
- Supports common operations: insertion, deletion, searching, sorting, and iteration.
- Offers built-in algorithms (like sorting and searching) through Collections utility class.
- Reduces programming effort by providing reusable and well-tested code.
- Ensures better performance and flexibility compared to traditional arrays.

List

Common ArrayList Operations

1. **add(element)** → Insert an element.
2. **get(index)** → Retrieve element at given position.
3. **set(index, element)** → Update/replace element at given position.
4. **remove(index/element)** → Delete element by index or value.
5. **size()** → Find number of elements.
6. **contains(element)** → Check if element exists.
7. **clear()** → Remove all elements.
8. **iteration** → Loop through elements using for or for-each.

```
import java.util.ArrayList;
public class ArrayListExample {
    public static void main(String[] args) {
        // Create ArrayList
        ArrayList<String> fruits = new ArrayList<>();

        // Add elements
        fruits.add("Apple");
```

```

fruits.add("Banana");
fruits.add("Mango");
fruits.add("Orange");
System.out.println("Fruits: " + fruits);

// Get element
System.out.println("Element at index 2: " + fruits.get(2));

// Set (Update) element
fruits.set(1, "Grapes");
System.out.println("After update: " + fruits);

// Remove element
fruits.remove("Apple");
System.out.println("After removing Apple: " + fruits);

// Check contains
System.out.println("Contains Mango? " + fruits.contains("Mango"));

// Size of list
System.out.println("Size: " + fruits.size());

// Iterate through elements
System.out.print("Iterating: ");
for(String fruit : fruits){
    System.out.print(fruit + " ");
}
}
}

```

Sample output:

```

Fruits: [Apple, Banana, Mango, Orange]
Element at index 2: Mango
After update: [Apple, Grapes, Mango, Orange]
After removing Apple: [Grapes, Mango, Orange]
Contains Mango? true
Size: 3
Iterating: Grapes Mango Orange

```

LinkedList

Common LinkedList Operations

1. **add(element)** → Add element at the end.
2. **addFirst(element)** → Insert element at the beginning.
3. **addLast(element)** → Insert element at the end.
4. **get(index)** → Retrieve element at specific index.
5. **set(index, element)** → Update/replace element at given index.
6. **remove(index/element)** → Remove element by index or value.
7. **removeFirst() / removeLast()** → Remove first or last element.
8. **size()** → Get number of elements.
9. **contains(element)** → Check if element exists.
10. **iteration** → Traverse the list using loops.

```

import java.util.LinkedList;

public class LinkedListExample {
    public static void main(String[] args) {
        // Create LinkedList
        LinkedList<String> animals = new LinkedList<>();
    }
}

```

```

// Add elements
animals.add("Dog");
animals.add("Cat");
animals.add("Horse");
System.out.println("Animals: " + animals);

// Add at first and last
animals.addFirst("Lion");
animals.addLast("Tiger");
System.out.println("After addFirst & addLast: " + animals);

// Get element
System.out.println("Element at index 2: " + animals.get(2));

// Set element
animals.set(1, "Elephant");
System.out.println("After update: " + animals);

// Remove element
animals.remove("Cat");
animals.removeFirst();
animals.removeLast();
System.out.println("After removals: " + animals);

// Contains
System.out.println("Contains Horse? " + animals.contains("Horse"));

// Size
System.out.println("Size: " + animals.size());

// Iteration
System.out.print("Iterating: ");
for(String animal : animals){
    System.out.print(animal + " ");
}
}
}

```

Sample output:

Animals: [Dog, Cat, Horse]
 After addFirst & addLast: [Lion, Dog, Cat, Horse, Tiger]
 Element at index 2: Cat
 After update: [Lion, Elephant, Cat, Horse, Tiger]
 After removals: [Elephant, Cat, Horse]
 Contains Horse? true
 Size: 3
 Iterating: Elephant Cat Horse

SET

Hash Set

Note: The order of elements may vary in HashSet because it does not maintain insertion order.

Common HashSet Operations

1. **add(element)** → Insert element (ignores duplicates).
2. **remove(element)** → Delete a specific element.
3. **contains(element)** → Check if an element exists.
4. **size()** → Get the number of elements.
5. **isEmpty()** → Check if the set is empty.
6. **clear()** → Remove all elements.
7. **iteration** → Traverse elements using a loop (order not guaranteed).

```
import java.util.HashSet;
```

```
public class HashSetExample {  
    public static void main(String[] args) {  
        // Create HashSet  
        HashSet<String> colors = new HashSet<>();  
  
        // Add elements  
        colors.add("Red");  
        colors.add("Green");  
        colors.add("Blue");  
        colors.add("Yellow");  
        colors.add("Red"); // duplicate, will be ignored  
        System.out.println("Colors: " + colors);  
  
        // Remove element  
        colors.remove("Green");  
        System.out.println("After removing Green: " + colors);  
  
        // Check if contains  
        System.out.println("Contains Blue? " + colors.contains("Blue"));  
  
        // Size  
        System.out.println("Size: " + colors.size());  
  
        // Check empty  
        System.out.println("Is empty? " + colors.isEmpty());  
  
        // Iteration  
        System.out.print("Iterating: ");  
        for (String c : colors) {  
            System.out.print(c + " ");  
        }  
    }  
}
```

Sample output:

```
Colors: [Red, Green, Blue, Yellow]  
After removing Green: [Red, Blue, Yellow]  
Contains Blue? true  
Size: 3  
Is empty? false  
Iterating: Red Blue Yellow
```

Tree Set

Common TreeSet Operations

1. **add(element)** → Insert element (keeps sorted order, ignores duplicates).
2. **remove(element)** → Delete a specific element.
3. **contains(element)** → Check if an element exists.
4. **size()** → Get number of elements.
5. **isEmpty()** → Check if the set is empty.
6. **first()** / **last()** → Get the smallest or largest element.
7. **higher(element)** / **lower(element)** → Get next greater or smaller element.
8. **clear()** → Remove all elements.
9. **iteration** → Traverse in **ascending sorted order**.

```
import java.util.TreeSet;
```

```
public class TreeSetExample {  
    public static void main(String[] args) {  
        // Create TreeSet  
        TreeSet<Integer> numbers = new TreeSet<>();  
  
        // Add elements  
        numbers.add(50);  
        numbers.add(10);  
        numbers.add(30);  
        numbers.add(20);  
        numbers.add(40);  
        numbers.add(30); // duplicate, ignored  
        System.out.println("Numbers: " + numbers);  
  
        // Remove element  
        numbers.remove(20);  
        System.out.println("After removing 20: " + numbers);  
  
        // Contains  
        System.out.println("Contains 30? " + numbers.contains(30));  
  
        // First and Last  
        System.out.println("First element: " + numbers.first());  
        System.out.println("Last element: " + numbers.last());  
  
        // Higher and Lower  
        System.out.println("Number higher than 30: " + numbers.higher(30));  
        System.out.println("Number lower than 30: " + numbers.lower(30));  
  
        // Size  
        System.out.println("Size: " + numbers.size());  
  
        // Iteration  
        System.out.print("Iterating: ");  
        for (Integer n : numbers) {  
            System.out.print(n + " ");  
        }  
    }  
}
```

Sample output:

Numbers: [10, 20, 30, 40, 50]
After removing 20: [10, 30, 40, 50]
Contains 30? true
First element: 10
Last element: 50
Number higher than 30: 40
Number lower than 30: 10
Size: 4
Iterating: 10 30 40 50

Maps

HashMap

Note:

- **Keys must be unique**, but **values can be duplicated**.
- Insertion order is **not guaranteed** (use LinkedHashMap for order, or TreeMap for sorted keys).

Common HashMap Operations

1. **put(key, value)** → Insert or update a key-value pair.
2. **get(key)** → Retrieve value for a given key.
3. **remove(key)** → Delete entry for a key.
4. **containsKey(key)** → Check if a key exists.
5. **containsValue(value)** → Check if a value exists.
6. **size()** → Number of key-value pairs.
7. **isEmpty()** → Check if map is empty.
8. **clear()** → Remove all entries.
9. **keySet() / values() / entrySet()** → Get all keys, values, or key-value pairs.
10. **iteration** → Traverse map entries using a loop.

```
import java.util.HashMap;
```

```
public class HashMapExample {  
    public static void main(String[] args) {  
        // Create HashMap  
        HashMap<Integer, String> students = new HashMap<>();  
  
        // Put elements  
        students.put(101, "Alice");  
        students.put(102, "Bob");  
        students.put(103, "Charlie");  
        students.put(104, "David");  
        students.put(102, "Bobby"); // updates value for key 102  
        System.out.println("Students: " + students);  
  
        // Get value  
        System.out.println("Student with ID 103: " + students.get(103));  
  
        // Remove element  
        students.remove(104);  
        System.out.println("After removing ID 104: " + students);  
  
        // Check key and value  
        System.out.println("Contains key 101? " + students.containsKey(101));  
        System.out.println("Contains value 'Charlie'? " + students.containsValue("Charlie"));  
  
        // Size
```

```

        System.out.println("Size: " + students.size());

        // Iterate entries
        System.out.println("Iterating key-value pairs:");
        for (Integer id : students.keySet()) {
            System.out.println("ID: " + id + ", Name: " + students.get(id));
        }
    }
}

```

Sample output:

```

Students: {101=Alice, 102=Bobby, 103=Charlie, 104=David}
Student with ID 103: Charlie
After removing ID 104: {101=Alice, 102=Bobby, 103=Charlie}
Contains key 101? true
Contains value 'Charlie'? true
Size: 3
Iterating key-value pairs:
ID: 101, Name: Alice
ID: 102, Name: Bobby
ID: 103, Name: Charlie

```

TreeMap

Notet:

Unlike HashMap, a TreeMap always keeps keys in sorted order.

Common TreeMap Operations

1. **put(key, value)** → Insert or update a key-value pair (keys sorted in ascending order by default).
2. **get(key)** → Retrieve value for a given key.
3. **remove(key)** → Delete entry for a key.
4. **containsKey(key) / containsValue(value)** → Check if a key or value exists.
5. **firstKey() / lastKey()** → Get the smallest or largest key.
6. **higherKey(key) / lowerKey(key)** → Get next higher or lower key.
7. **size()** → Number of key-value pairs.
8. **clear()** → Remove all entries.
9. **keySet() / values() / entrySet()** → Get all keys, values, or entries.
10. **iteration** → Traverse map in **sorted key order**.

```
import java.util.TreeMap;
```

```

public class TreeMapExample {
    public static void main(String[] args) {
        // Create TreeMap
        TreeMap<Integer, String> students = new TreeMap<>();

        // Put elements (keys auto-sorted)
        students.put(103, "Charlie");
        students.put(101, "Alice");
        students.put(104, "David");
        students.put(102, "Bob");
        System.out.println("Students: " + students);

        // Get value
        System.out.println("Student with ID 102: " + students.get(102));

        // Remove element
        students.remove(104);
    }
}

```

```

System.out.println("After removing ID 104: " + students);

// Check contains
System.out.println("Contains key 101? " + students.containsKey(101));
System.out.println("Contains value 'Charlie'? " + students.containsValue("Charlie"));

// First and Last key
System.out.println("First Key: " + students.firstKey());
System.out.println("Last Key: " + students.lastKey());

// Higher and Lower key
System.out.println("Higher than 102: " + students.higherKey(102));
System.out.println("Lower than 102: " + students.lowerKey(102));

// Iteration (sorted order)
System.out.println("Iterating key-value pairs:");
for (Integer id : students.keySet()) {
    System.out.println("ID: " + id + ", Name: " + students.get(id));
}
}
}

```

Sample output:

```

Students: {101=Alice, 102=Bob, 103=Charlie, 104=David}
Student with ID 102: Bob
After removing ID 104: {101=Alice, 102=Bob, 103=Charlie}
Contains key 101? true
Contains value 'Charlie'? true
First Key: 101
Last Key: 103
Higher than 102: 103
Lower than 102: 101
Iterating key-value pairs:
ID: 101, Name: Alice
ID: 102, Name: Bob
ID: 103, Name: Charlie

```

(Dear students refer documents of 21st and 22nd classes for detailed explanation)

Students To Address Following JCF Based Programming Concepts

7.1: Implementation of ArrayList and LinkedList for Student Management

Aim: To implement a Student Management System using Java's ArrayList and LinkedList, and perform operations such as insertion, deletion, searching, updating, traversal, sorting, and list-specific operations.

Scenario

You are developing a Student Management System.

- ArrayList is used to maintain the list of Active Students, where frequent retrieval and indexed access are required.
- LinkedList is used to maintain the list of Alumni Students, where insertion and deletion at the beginning or end of the list are required.

The system should support adding, removing, searching, updating, traversing, and managing student records.

Part A — Active Students Using ArrayList Tasks / Operations Students must perform the following operations: 1. Create an ArrayList of student names. 2. Add 6 active students. 3. Add a student at a specific position. 4. Add multiple students using addAll(). 5. Display all active students. 6. Access a student using its index. 7. Update a student's name. 8. Check whether a student exists. 9. Find the index of a student. 10. Find the last occurrence of a student. 11. Remove a student by name. 12. Remove a student by index. 13. Find the number of students. 14. Check whether the list is empty. 15. Iterate using an enhanced for loop. 16. Iterate using forEach(). 17. Sort students alphabetically. 18. Reverse the list. 19. Create a sublist. 20. Clear the list and verify that it is empty.	Part B — Alumni Students Using LinkedList Tasks / Operations Students must perform the following: 1. Create a LinkedList of alumni names. 2. Add 6 alumni students. 3. Add an alumnus at the beginning. 4. Add an alumnus at the end. 5. Add an alumnus at a specific position. 6. Access the first alumnus. 7. Access the last alumnus. 8. Access an alumnus by index. 9. Update an alumnus. 10. Search for an alumnus. 11. Find the index of an alumnus. 12. Remove an alumnus by name. 13. Remove the first alumnus. 14. Remove the last alumnus. 15. Retrieve the first element without removing it. 16. Retrieve the last element without removing it. 17. Retrieve and remove the first element. 18. Retrieve and remove the last element. 19. Iterate through the list. 20. Display the list in reverse order. 21. Check size and emptiness. 22. Clear the list.
---	---

Important ArrayList Methods		Important LinkedList Methods	
Method	Purpose	Method	Purpose
add()	Adds an element	add()	Adds an element
add(index, element)	Inserts at a specified position	add(index, element)	Adds at specified position
addAll()	Adds multiple elements	addFirst()	Adds at beginning
get()	Retrieves an element	addLast()	Adds at end
set()	Updates an element	offer()	Adds element
remove()	Removes an element	offerFirst()	Adds at beginning
removeAll()	Removes matching elements	offerLast()	Adds at end

contains()	Checks existence	get()	Gets element by index
indexOf()	Finds first occurrence	getFirst()	Gets first element
lastIndexOf()	Finds last occurrence	getLast()	Gets last element
size()	Returns number of elements	peek()	Retrieves first element
isEmpty()	Checks whether list is empty	peekFirst()	Retrieves first element
clear()	Removes all elements	peekLast()	Retrieves last element
sort()	Sorts elements	poll()	Retrieves and removes first
subList()	Extracts a portion of list	pollFirst()	Retrieves and removes first
forEach()	Iterates through elements	pollLast()	Retrieves and removes last
		set()	Updates element
		remove()	Removes element
		removeFirst()	Removes first element
		removeLast()	Removes last element
		contains()	Checks existence
		indexOf()	Finds first occurrence
		lastIndexOf()	Finds last occurrence
		size()	Returns number of elements
		isEmpty()	Checks emptiness
		clear()	Removes all elements
		descendingIterator()	Traverses in reverse

Sample Output

For ArrayList: Active Students

Active Students: [Arjun, Bhavana, Charan, Divya, Farhan, Gowri]

After inserting Deepak:

[Arjun, Bhavana, Deepak, Charan, Divya, Farhan, Gowri]

After adding multiple students:

[Arjun, Bhavana, Deepak, Charan, Divya, Farhan, Gowri, Harish, Isha]

Student at index 3: Charan

After updating student:

[Arjun, Bhavana, Deepak, Divya Sharma, Divya, Farhan, Gowri, Harish, Isha]

Contains Arjun? true

Contains Kiran? false

Index of Bhavana: 1

After adding duplicate Arjun:

[Arjun, Bhavana, Deepak, Divya Sharma, Divya, Farhan, Gowri, Harish, Isha, Arjun]

Last index of Arjun: 9

After removing Charan:

...

Number of Active Students: 8

All Active Students:

Arjun

Bhavana

Deepak

Divya Sharma
Divya
Farhan
Gowri
Harish
Isha

Sorted Students:
[...]

Reverse Order:
[...]

Is Empty? false

After clearing:
Students: []
Is Empty? True

Output for LinkedList: Alumni Students

Alumni Students: [Rahul, Sneha, Vikram, Ananya, Karthik, Meera]

After adding first and last: [Aditya, Rahul, Sneha, Vikram, Ananya, Karthik, Meera, Nisha]

After inserting Rohan: [Aditya, Rahul, Rohan, Sneha, Vikram, Ananya, Karthik, Meera, Nisha]

First Alumni: Aditya
Last Alumni: Nisha
Alumni at index 3: Sneha

After updating: [Aditya, Rahul, Rohan, Ananya Rao, Vikram, Ananya, Karthik, Meera, Nisha]

Contains Rahul? true
Contains Suresh? false

Index of Sneha: -1

After removing Vikram: [Aditya, Rahul, Rohan, Ananya Rao, Ananya, Karthik, Meera, Nisha]
After removing first: [Rahul, Rohan, Ananya Rao, Ananya, Karthik, Meera, Nisha]
After removing last: [Rahul, Rohan, Ananya Rao, Ananya, Karthik, Meera]

First using peek(): Rahul
Peek First: Rahul
Peek Last: Meera

Removed using pollFirst(): Rahul
Removed using pollLast(): Meera
Remaining Alumni: [Rohan, Ananya Rao, Ananya, Karthik]

Number of Alumni: 4

All Alumni:
Rohan
Ananya Rao
Ananya
Karthik

Using Iterator:

Rohan

Ananya Rao

Ananya

Karthik

Reverse Order:

Karthik

Ananya

Ananya Rao

Rohan

Is Empty? false

After clearing:

Alumni: []

Is Empty? true

7.2.: Implementation of HashSet and TreeSet for Managing Unique and Sorted Data

Aim

To implement and analyze the operations of **HashSet** and **TreeSet** in Java for storing unique elements and maintaining sorted elements respectively.

Learning Objectives

After completing this experiment, students should be able to:

1. Create and initialize HashSet and TreeSet.
2. Add and remove elements.
3. Handle duplicate elements.
4. Search for elements.
5. Determine collection size.
6. Iterate through collections.
7. Clear collections and verify emptiness.
8. Apply set operations such as union, intersection, and difference.
9. Use TreeSet navigation methods such as higher(), lower(), ceiling(), and floor().
10. Retrieve minimum and maximum elements.

Part A — HashSet: Unique Product Names

Scenario: You are developing a small inventory system. The system must maintain a collection of unique product names. Since duplicate product names should not be stored, use a `HashSet<String>`.

Requirements

Perform the following operations:

1. Create a HashSet to store product names.
2. Add six product names, including duplicate names.
3. Display the set.
4. Remove a product.
5. Check whether a particular product exists.
6. Find the number of products.
7. Iterate through all products.
8. Check whether the set is empty.
9. Clear the set.
10. Verify that the set is empty.
11. Demonstrate adding all elements from another set.
12. Demonstrate set union, intersection, and difference.

Note: The exact order of elements in a HashSet is not guaranteed. Therefore, students should not expect the displayed order to always match the sample output.

HashSet methods:

Method	Purpose
<code>add()</code>	Adds an element
<code>addAll()</code>	Adds all elements from another collection
<code>remove()</code>	Removes a specified element
<code>removeAll()</code>	Removes elements present in another collection
<code>retainAll()</code>	Retains only common elements
<code>contains()</code>	Checks whether an element exists
<code>containsAll()</code>	Checks whether all specified elements exist
<code>size()</code>	Returns number of elements
<code>isEmpty()</code>	Checks whether the set is empty
<code>clear()</code>	Removes all elements
<code>iterator()</code>	Creates an iterator
<code>forEach()</code>	Iterates through elements

Expected Result

The duplicate products are automatically ignored.

Products: [Laptop, Mouse, Tablet, Phone, Keyboard]

After removing Keyboard: [Laptop, Mouse, Tablet, Phone]

Contains Laptop? true

Contains Printer? false

Number of Products: 4

All Products:

Laptop

Mouse

Tablet

Phone

Using forEach():

Laptop

Mouse

Tablet

Phone

Is Empty? false

After clearing, is empty? true

Part B — TreeSet: Sorted Student Scores

Scenario: You are maintaining student examination scores. The scores must be:

- unique, and
- automatically maintained in ascending order.

Use a `TreeSet<Integer>`.

Requirements

1. Create a `TreeSet` for student scores.
2. Add six scores, including duplicates.
3. Display the sorted scores.
4. Remove a score.
5. Retrieve the lowest score.
6. Retrieve the highest score.
7. Find the score immediately higher than a given score.
8. Find the score immediately lower than a given score.
9. Check whether a score exists.
10. Iterate through scores in ascending order.
11. Find the first and last scores.
12. Demonstrate `ceiling()` and `floor()`.
13. Demonstrate `pollFirst()` and `pollLast()`.
14. Demonstrate descending-order traversal.

Important TreeSet Methods

Method	Purpose
<code>add()</code>	Adds an element
<code>remove()</code>	Removes an element
<code>contains()</code>	Checks whether an element exists

size()	Returns number of elements
isEmpty()	Checks whether empty
clear()	Removes all elements
first()	Returns smallest element
last()	Returns largest element
higher()	Returns element strictly greater than given value
lower()	Returns element strictly smaller than given value
ceiling()	Returns element greater than or equal to given value
floor()	Returns element less than or equal to given value
pollFirst()	Retrieves and removes smallest element
pollLast()	Retrieves and removes largest element
descendingSet()	Returns elements in descending order
headSet()	Returns elements smaller than a specified value
tailSet()	Returns elements greater than/equal to a specified value
subSet()	Returns elements within a specified range

Expected Output

Scores: [75, 79, 85, 88, 92]

After removing 75: [79, 85, 88, 92]

Lowest Score: 79

Highest Score: 92

Score higher than 85: 88

Score lower than 85: 79

Contains 88? true

Score greater than or equal to 86: 88

Score less than or equal to 86: 85

Number of Scores: 4

All Scores:

79 85 88 92

Scores in Descending Order:

92 88 85 79

Scores less than 88: [79, 85]

Scores greater than or equal to 85: [85, 88, 92]

Scores from 79 to 92: [79, 85, 88, 92]

Removed Lowest Score: 79

Removed Highest Score: 92

Remaining Scores: [85, 88]

7.3: Implementation of HashMap and TreeMap for Employee Record Management

Aim: To implement an Employee Management System using Java's HashMap and TreeMap, and perform operations such as insertion, retrieval, updating, deletion, searching, iteration, and key-based navigation.

Scenario

You are managing employee records using:

- **Employee ID** → Integer
- **Employee Name** → String

Two different map implementations are required:

HashMap

Use HashMap<Integer, String> when the primary requirement is key-based lookup without requiring sorted order.

TreeMap

Use TreeMap<Integer, String> when employee records must be maintained in ascending order of Employee ID and navigation based on keys is required.

Learning Objectives

After completing this experiment, should be able to:

1. Create and initialize HashMap and TreeMap.
2. Store key-value pairs using put().
3. Retrieve values using get().
4. Update existing values.
5. Remove records using keys.
6. Search using keys and values.
7. Determine map size.
8. Check whether a map is empty.
9. Remove all records.
10. Traverse keys, values, and entries.
11. Retrieve the first and last keys from a TreeMap.
12. Find higher and lower keys.
13. Compare the behavior of HashMap and TreeMap.
14. Select an appropriate map implementation for a given application scenario.

Part A — HashMap Employee Records

Tasks / Operations

Implement the following operations:

1. Create a HashMap<Integer, String>.
2. Add four employee records using put().
3. Retrieve an employee name using Employee ID.
4. Update the name of an existing employee.
5. Remove an employee using Employee ID.
6. Check whether a particular Employee ID exists.
7. Check whether a particular employee name exists.
8. Find the number of employee records.
9. Check whether the map is empty.
10. Display all Employee IDs using keySet().
11. Display all employee names using values().
12. Display all employee ID-name pairs using entrySet().
13. Clear the map.
14. Verify that the map is empty.

Part B — TreeMap Employee Records

Tasks / Operations

Implement the following:

1. Create a TreeMap<Integer, String>.
2. Add four employee records.
3. Observe automatic sorting based on Employee ID.
4. Retrieve an employee using get().
5. Update an employee using put().
6. Remove an employee.
7. Check whether an Employee ID exists.
8. Check whether an employee name exists.
9. Find the smallest Employee ID.
10. Find the largest Employee ID.
11. Find the Employee ID immediately higher than a given ID.
12. Find the Employee ID immediately lower than a given ID.
13. Find the Employee ID greater than or equal to a specified ID.

	14. Find the Employee ID less than or equal to a specified ID. 15. Find the number of employees. 16. Iterate using keySet(). 17. Iterate using values(). 18. Iterate using entrySet(). 19. Display entries in descending order. 20. Clear the map and verify emptiness.
--	---

HashMap Methods

Method	Purpose
put()	Adds or updates a key-value pair
get()	Retrieves value using key
remove()	Removes a mapping
containsKey()	Checks whether a key exists
containsValue()	Checks whether a value exists
size()	Returns number of mappings
isEmpty()	Checks whether map is empty
clear()	Removes all mappings
keySet()	Returns all keys
values()	Returns all values
entrySet()	Returns all key-value pairs
getOrDefault()	Returns value or default value
putIfAbsent()	Adds only if key does not already exist
replace()	Replaces value for an existing key
replaceAll()	Replaces values using a function

TreeMap Methods

Method	Purpose
put()	Adds or updates mapping
get()	Retrieves value using key
remove()	Removes mapping
containsKey()	Checks key
containsValue()	Checks value
size()	Number of mappings
isEmpty()	Checks emptiness
clear()	Removes all mappings
firstKey()	Returns smallest key
lastKey()	Returns largest key
higherKey()	Returns key strictly greater
lowerKey()	Returns key strictly smaller
ceilingKey()	Returns key $\geq$ specified key
floorKey()	Returns key $\leq$ specified key
keySet()	Returns keys in sorted order
values()	Returns values according to key order
entrySet()	Returns entries in key order
descendingKeySet()	Returns keys in descending order
descendingMap()	Returns map in reverse key order
headMap()	Returns entries before a key
tailMap()	Returns entries from a key
subMap()	Returns entries within a key range

Sample output HashMap::

HashMap:

{101=Bob, 102=Bobby, 103=Alice, 104=Charlie}

Employee 103: Alice

After updating Employee 102:

{101=Bob, 102=Bobby Kumar, 103=Alice, 104=Charlie}

After removing Employee 104:

{101=Bob, 102=Bobby Kumar, 103=Alice}

Contains ID 101? true

Contains 'Charlie'? true

Number of Employees: 4

Is HashMap empty? false

Employee IDs:

101

102

103

104

Employee Names:

Bob

Bobby Kumar

Alice

Charlie

HashMap Entries:

ID: 101, Name: Bob

ID: 102, Name: Bobby Kumar

ID: 103, Name: Alice

ID: 104, Name: Charlie

Employee 999: Employee Not Found

After putIfAbsent():

...

After replace():

...

After clearing, is empty? true

Sample output TreeMap::

TreeMap:

{101=Bob, 102=Bobby, 103=Alice, 104=Charlie}

Employee 104: Charlie

After updating Employee 102:

{101=Bob, 102=Bobby Kumar, 103=Alice, 104=Charlie}

After removing Employee 104:

{101=Bob, 102=Bobby Kumar, 103=Alice}

Contains ID 101? true

Contains 'Charlie'? true

First ID: 101

Last ID: 104

Higher than 102: 103

Lower than 102: 101

Ceiling of 102: 102

Floor of 102: 102

Number of Employees: 4

Employee IDs:

101

102

103

104

Employee Names:

Bob

Bobby Kumar

Alice

Charlie

TreeMap Entries:

ID: 101, Name: Bob

ID: 102, Name: Bobby Kumar

ID: 103, Name: Alice

ID: 104, Name: Charlie

Employee IDs in Descending Order:

104

103

102

101

Descending Map:

{104=Charlie, 103=Alice, 102=Bobby Kumar, 101=Bob}

After clearing, is empty? true